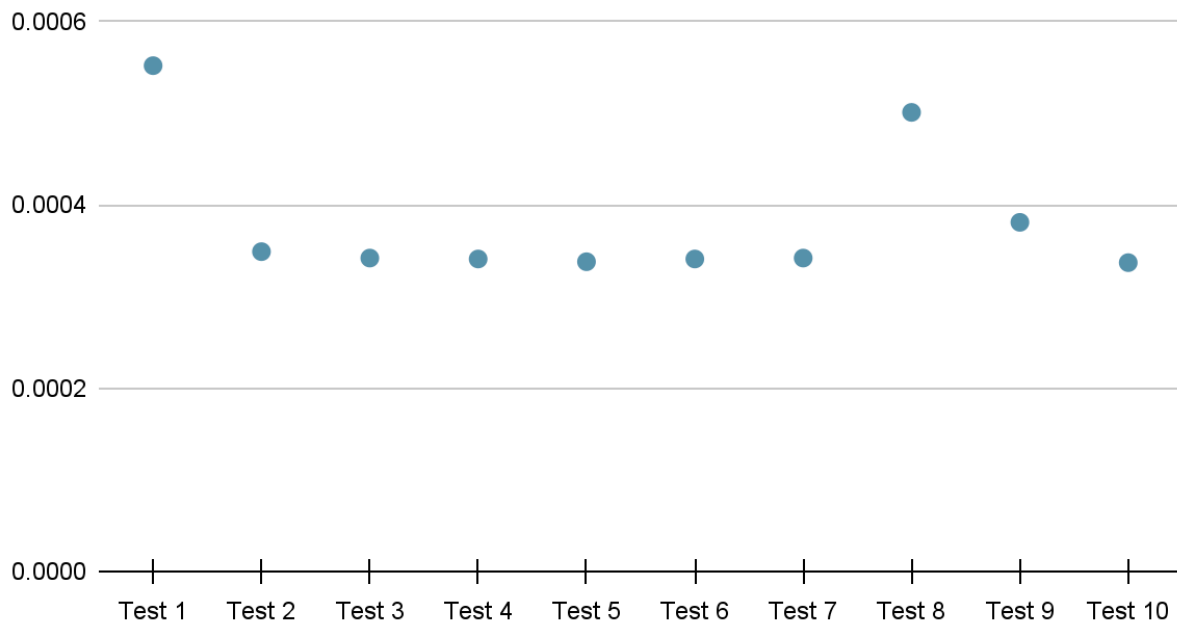


Question 1:

*Use at least 10 nontrivial input files (i.e., contain strings of length at least 25).
Graph the runtime of the 10 files.*

Time to Run HVLCS (Seconds)



We only measured the time it takes to run the HVLCS algorithm we wrote. We did not include file reading time as it should be somewhat consistent (with the same parameters)

String for record keeping purposes: [0.000552, 0.000349, 0.000342, 0.000341, 0.000338, 0.000341, 0.000342, 0.000501, 0.000381, 0.000337]

Question 2:

Give a recurrence that is the basis of a dynamic programming algorithm to compute the HVLCS of strings A and B. You must provide the appropriate base cases, and explain why your recurrence is correct.

Highest Value Longest Common Sequence

$$\text{OPT}(i, j) = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ \max(\text{OPT}(i-1, j), \text{OPT}(i, j-1), v(a_i) + \text{OPT}(i-1, j-1)) & \text{if } a_i = b_j \\ \max(\text{OPT}(i-1, j), \text{OPT}(i, j-1)) & \text{if } a_i \neq b_j \end{cases}$$

The recurrence equation above computes the basis of the dynamic programming algorithm. i is the index in the A string, j is the index in the B string.

$\text{OPT}(i-1, j)$ means skipping a char in A, $\text{OPT}(i, j-1)$ means skipping a char in B, and $v(a_i) + \text{OPT}(i-1, j-1)$, means putting the character into the subsequence (selecting both).

The recurrence is correct in that it computes the optimal solutions for subproblems, and then uses these optimal solutions to compute larger solutions until the final solution is reached. Therefore it follows the principle of induction, in that the base cases where $i=0$ and $j=0$ are optimal trivially, the cases that follow from there where $i=1$ or $j=1$ are also optimal since the base cases are optimal, and so on and so forth inductively until the final $\text{OPT}(i, j)$ is reached.

Question 3:

Give pseudocode of an algorithm to compute the length of the HV LCS of given strings A and B. What is the runtime of your algorithm?

The question did not say whether the dp table has already been filled as a prerequisite for backtracking so we are assuming that it has not.

Inputs: String A of length m , String B of length n , and value table/function $v(c) = V$, where c is the character and V is the associated value

$\text{dp}[m+1][n+1]$

for $i = 0$ to m :

$\text{dp}[i][0] = 0$

for $i = 0$ to n :

$\text{dp}[0][i] = 0$

// Finding optimal value (fill dp table)

for $i = 1$ to m :

 for $j = 1$ to n :

Highest Value Longest Common Sequence

```

if A[i - 1] == B[j - 1]:
    dp[i][j] = dp[i - 1][j - 1] + v(A[i - 1])
else:
    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

```

```
// Finding length
```

```
Result = "", i = m, j = n
```

```

while i > 0 AND j > 0:
    if A[i - 1] == B[j - 1]:
        result = A[i - 1] + result
        i = i - 1
        j = j - 1
    else if dp[i - 1][j] > dp[i][j - 1]:
        i = i - 1
    else
        j = j - 1
Return len(result)

```

Runtime is $O(m * n)$ where m is the length of String A and n is the length of String B.

Filling the table is $O(m * n)$ while backtracking is $O(m + n)$. The result is $O(m * n + m + n)$, which simplifies to $O(m * n)$.

If we are only backtracking then the runtime would be $O(m + n)$ where m is the length of String A and n is the length of String B, but this is not stated in the problem.